

Faculdade de Engenharia da Universidade do Porto



Assignment 2

Multigraph-Based Prototype and Empirical Evaluation

Routing, Priority Queues, Spatial Indexing, and Graph Coloring

Course

Advanced Data Structures and Algorithms

Academic Year 2025/2026

Group 8

Students Ângelo Oliveira
José Lopes
Mário Araújo

Professor Jácome Cunha

Porto, June 9, 2026

Abstract

This report presents a multigraph-based prototype for solving routing problems in a multi-modal transport network. The initial goal was to model and solve shortest-path routing where multiple transportation modes can connect the same pair of locations. The resulting graph contains walking, bus, metro, and transfer edges, each weighted by travel time.

After implementing routing, the prototype was extended to evaluate additional graph algorithms and data structures, including Dijkstra, A*, Prim, brute-force and heap-based priority queues, interference graph construction using brute force and Quadtrees, and graph coloring using brute force and Welsh–Powell. The report combines theoretical analysis, implementation details, pseudocode, and empirical benchmarks.

The main conclusion is that asymptotic complexity alone is not sufficient to predict practical performance: memory layout, cache locality, graph density, heuristic quality, and structural overhead strongly influence the final result.

Contents

Abstract	i
1 Introduction	1
1.1 Main Contributions	1
1.2 Report Organization	1
2 Data Construction	2
3 Multigraph Representation	2
3.1 Internal Representation	3
4 Benchmark Methodology	3
5 Shortest Path Routing	4
5.1 Dijkstra on a Multigraph	4
5.2 A* Search	5
5.3 Benchmark: Dijkstra vs A*	5
6 Priority Queues	7
6.1 Theoretical Comparison	8
6.2 Brute-Force Priority Queue	8
6.3 Mutable Binary Heap	8
6.4 Fibonacci Heap	9
6.5 Benchmark: Priority Queues	9
7 Graph Coloring	12
7.1 Brute-Force Coloring	12
7.2 Welsh–Powell Heuristic	13
7.3 Benchmark: Coloring	13
8 Interference Graph and Quadtree	14
8.1 Brute-Force Interference Graph	14
8.2 Quadtree-Based Construction	15
8.3 Benchmark: Quadtree vs Brute Force	15
9 Discussion	16
9.1 Theory vs Practice	17
9.2 The Role of Heuristics	17
9.3 The Importance of Data Structures	17
10 Conclusion	18
A Prim Pseudocode	18

1 Introduction

Routing problems are central to transportation, logistics, and network optimization. A good routing algorithm can reduce travel time, operational cost, energy usage, and unnecessary transfers. In real urban mobility, however, a transportation network is rarely a simple graph: the same two locations may be connected by walking, bus, metro, or transfer movements, each with different cost and meaning.

The primary problem addressed in this project was therefore:

How can we solve routing problems in a multigraph?

A multigraph is appropriate because multiple edges may exist between the same pair of vertices. In our case, those edges represent different transportation modes. The graph contains four main edge types:

- **WALK**: pedestrian movement through the street network;
- **BUS**: STCP bus connections;
- **METRO**: Metro do Porto connections;
- **TRANSFER**: movement between layers, including a boarding penalty.

After building the routing structure, the project expanded into a broader empirical study. We used the same graph framework to test other graph algorithms and data structures, including Prim’s algorithm, graph coloring, priority queues, Quadtrees, and heuristics such as A*. This extension was intentional: the goal was not only to implement algorithms, but also to understand how they behave in practice when executed on real hardware and on graphs of different sizes and densities.

1.1 Main Contributions

The main contributions of the prototype are:

- construction of a multi-modal transport multigraph for Porto;
- adaptation of shortest-path algorithms to a multigraph representation;
- implementation and comparison of brute-force, binary-heap, and Fibonacci-heap priority queues;
- implementation of A* using coordinates as a heuristic;
- implementation of Prim using different priority queues;
- construction of interference graphs using both brute force and Quadtree;
- implementation of brute-force coloring and the Welsh–Powell heuristic;
- empirical evaluation using time, memory, cache misses, operation counts, and correctness checks.

1.2 Report Organization

Section 2 describes how the transport data was built and normalized. Section 3 explains the multigraph representation. Section 4 summarizes the benchmark methodology. Section 5 discusses Dijkstra and A*, and Section 6 compares priority queues over Prim’s Algorithm. Section 7 presents graph coloring and Welsh–Powell. Section 8 explains the interference graph and the Quadtree approach. Section 9 discusses the main findings, and Section 10 concludes the report.

2 Data Construction

The data pipeline was designed to build a multi-modal public-transport graph for Porto, Portugal, combining pedestrian, bus, and metro networks into a single graph representation. The walk layer was extracted from OSMnx [5]. The bus network was built from STCP GTFS data [6] automatically downloaded from Open Data Porto [7]. Metro data were obtained from Metro do Porto GTFS feeds when available, with an OSMnx rail-network fallback.

Graph construction proceeded by downloading or loading cached data, extracting the walk network, building the bus and metro layers, creating transfer edges between layers, and checking connectivity. Nodes representing the same physical location were then merged using Union-Find, after which edge weights were converted from metres to seconds. Finally, the normalized graph was saved and diagnostic statistics were generated.

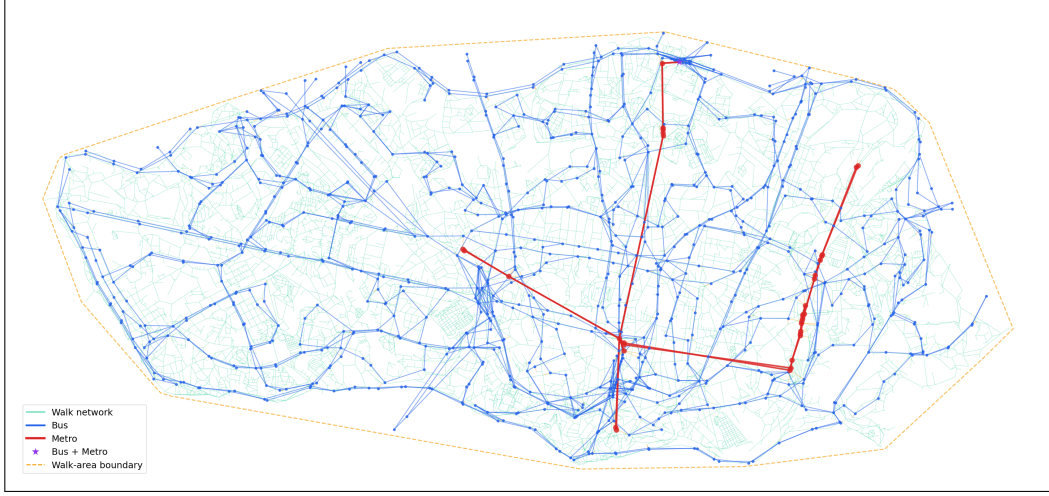


Figure 1: Normalized Porto multi-modal graph.

Travel time was used as the edge weight because minimizing physical distance is not equivalent to minimizing travel time. Without this conversion, shortest-path algorithms would often prefer walking routes simply because pedestrian edges are shorter in metres, even when public transport provides a faster journey. Edge weights were therefore computed as

$$\begin{aligned}
 w_{\text{walk}} &= \frac{\text{metres}}{1.4}, \\
 w_{\text{bus}} &= \frac{\text{metres}}{6.9}, \\
 w_{\text{metro}} &= \frac{\text{metres}}{11.1}, \\
 w_{\text{transfer}} &= \frac{\text{metres}}{1.4} + 180.
 \end{aligned} \tag{1}$$

The transfer penalty models the time required to board or change transportation modes and encourages routes that better reflect realistic travel behaviour.

3 Multigraph Representation

A simple graph is not expressive enough for this problem because the same two locations may be connected in multiple ways. For example, two nearby locations may have a walking connection and also be connected by bus or metro. These connections should not be merged into a single edge because they represent different costs and modes.

3.1 Internal Representation

The core structure is a **Multigraph**. Each vertex stores an adjacency list of edges. Each edge stores its origin, destination, weight, and transportation mode.

Structure	Stored Information
Vertex	id coordinates adjacency_list: [Edge] algorithm state: dist, isVisited, path
Edge	origin destination weight mode: WALK BUS METRO TRANSFER

Table 1: Internal representation of vertices and edges.

The key implementation detail is that algorithms iterate over outgoing edges, not just neighbouring vertices. This makes Dijkstra, A*, and Prim naturally compatible with parallel edges. If two vertices are connected by multiple modes, each edge is relaxed independently.

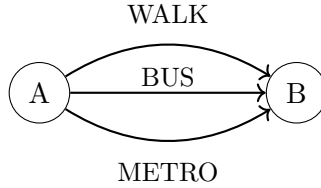


Figure 2: Example of parallel edges between two vertices in a multigraph.

4 Benchmark Methodology

All experiments were designed to be reproducible and comparable across implementations. Each benchmark used a fixed seed and configurable graph parameters, including graph size, average vertex degree, node count, and radius for spatial tests. The framework supported warm-up runs before measured repetitions, optional cache-miss measurement on supported systems, and correctness checks against reference implementations.

For each experiment, execution time, memory usage, and algorithm-specific statistics were collected. Execution time was measured in milliseconds. When available, cache references and cache misses were also recorded. Additional metrics depended on the algorithm under evaluation and included quantities such as insertions, deletions, decrease-key operations, distance checks, nodes pruned, checksums, coloring validity, and optimality comparisons.

Empirical evaluation was necessary because asymptotic complexity alone does not fully predict practical performance. Real-world execution is influenced by factors such as memory layout, cache locality, pointer overhead, allocation patterns, graph density, and constant factors hidden by asymptotic notation.

5 Shortest Path Routing

5.1 Dijkstra on a Multigraph

Dijkstra's algorithm computes shortest paths from a source to all reachable vertices, or until a destination is reached. In this project, it was used for routing in the multigraph.

The adaptation to a multigraph is straightforward: instead of assuming a single edge between two vertices, the algorithm relaxes every outgoing edge in the adjacency list. Therefore, if a vertex has multiple edges to the same destination with different modes, each is considered independently.

Algorithm 1 Dijkstra on a Multigraph

Require: Graph $G = (V, E)$, source s , destination t , priority queue Q

```
1: for all  $v \in V$  do
2:    $dist[v] \leftarrow \infty$ 
3:    $visited[v] \leftarrow false$ 
4:    $path[v] \leftarrow null$ 
5: end for
6:  $dist[s] \leftarrow 0$ 
7:  $Q.insert(s, 0)$ 
8: while  $Q$  is not empty do
9:    $u \leftarrow Q.extractMin()$ 
10:  if  $u = t$  then
11:    break
12:  end if
13:   $visited[u] \leftarrow true$ 
14:  for all edge  $e = (u, v)$  in  $adj[u]$  do
15:    if  $visited[v]$  then
16:      continue
17:    end if
18:     $newDist \leftarrow dist[u] + weight(e)$ 
19:    if  $newDist < dist[v]$  then
20:       $dist[v] \leftarrow newDist$ 
21:       $path[v] \leftarrow e$ 
22:      if  $v$  is already in  $Q$  then
23:         $Q.decreaseKey(v, newDist)$ 
24:      else
25:         $Q.insert(v, newDist)$ 
26:      end if
27:    end if
28:  end for
29: end while
30: return path from  $s$  to  $t$ 
```

During development, visualizing Dijkstra helped reveal an important limitation: Dijkstra has no notion of direction toward the destination. It expands the vertex with the smallest known distance from the source, even if that vertex appears geographically unintuitive from a human perspective.

This observation motivated the implementation of A*. Since vertices have coordinates, it is possible to use a geometric heuristic that estimates the remaining distance to the target.

5.2 A* Search

A* extends Dijkstra by changing the priority used in the queue:

$$f(n) = g(n) + h(n), \quad (2)$$

where $g(n)$ is the known cost from the source to node n , and $h(n)$ is an estimate of the remaining cost from n to the destination.

In this project, $h(n)$ was based on the Euclidean distance between coordinates divided by a speed value. The heuristic must not overestimate the true remaining cost if optimality is required.

Algorithm 2 A* on a Multigraph

Require: Graph $G = (V, E)$, source s , destination t , priority queue Q

```
1: for all  $v \in V$  do
2:    $g[v] \leftarrow \infty$ 
3:    $visited[v] \leftarrow false$ 
4:    $path[v] \leftarrow null$ 
5: end for
6:  $g[s] \leftarrow 0$ 
7:  $Q.insert(s, h(s, t))$ 
8: while  $Q$  is not empty do
9:    $u \leftarrow Q.extractMin()$ 
10:  if  $u = t$  then
11:    break
12:  end if
13:   $visited[u] \leftarrow true$ 
14:  for all edge  $e = (u, v)$  in  $adj[u]$  do
15:    if  $visited[v]$  then
16:      continue
17:    end if
18:     $tentativeG \leftarrow g[u] + weight(e)$ 
19:    if  $tentativeG < g[v]$  then
20:       $g[v] \leftarrow tentativeG$ 
21:       $path[v] \leftarrow e$ 
22:       $f[v] \leftarrow g[v] + h(v, t)$ 
23:      if  $v$  is already in  $Q$  then
24:         $Q.decreaseKey(v, f[v])$ 
25:      else
26:         $Q.insert(v, f[v])$ 
27:      end if
28:    end if
29:  end for
30: end while
31: return path from  $s$  to  $t$ 
```

5.3 Benchmark: Dijkstra vs A*

The benchmark compared Dijkstra and A* using the same graph generation seed, source, destination, and priority queue. Correctness was checked by comparing the final shortest-path distance. In addition to runtime, we also measured how many vertices were reached and processed by each algorithm, so that the performance difference could be explained by the amount of graph exploration.

Discussion. The results confirm the expected advantage of A* over Dijkstra when the graph contains meaningful spatial information. Dijkstra only uses the cost already accumulated from the source, so it expands vertices according to the currently known shortest distance. In contrast, A* combines this known cost with a heuristic estimate of the remaining distance to the destination which allows it to choose vertices that are more promising geographically.

This difference is visible both in the benchmark results and in the exploration plots. In the Porto visualization, Dijkstra reached 2,803 vertices and processed 2,284 vertices, while A* reached 1,518 vertices and processed 1,208 vertices for the same source and destination. Both algorithms found a path with 21 vertices, but A* explored a much smaller region of the graph before reaching the destination. This shows that Dijkstra expands more broadly around the source, while A* shows a clear bias toward the destination.

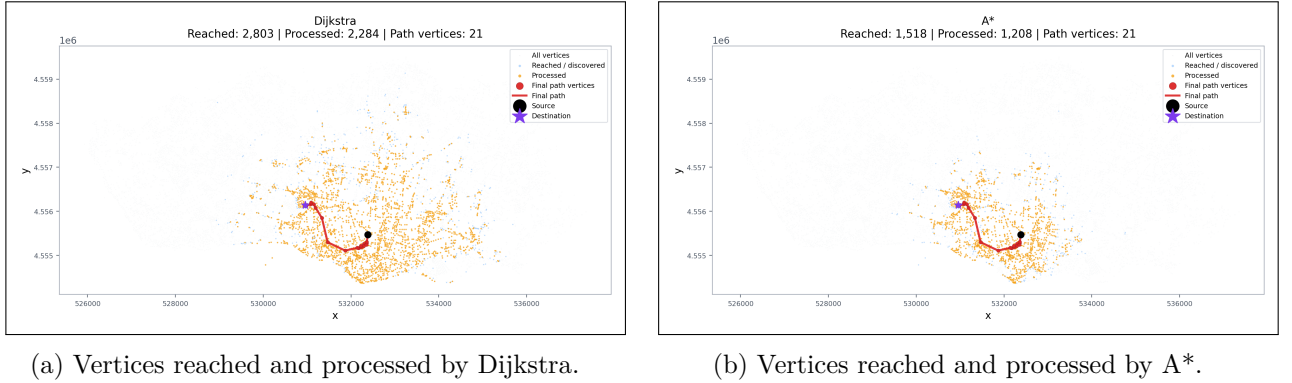


Figure 3: Exploration comparison on the Porto graph by Dijkstra and A*.

The runtime benchmark also supports this observation. A* was consistently faster than Dijkstra across the tested graph sizes and average degrees. Depending on the graph configuration, the measured speedup ranged from approximately $3.5\times$ to more than $12\times$.

The variation in the speedup is expected. Even for larger graphs, Dijkstra can sometimes finish relatively quickly if the destination is reached through a short path or if the target happens to be located in a region that is discovered early. Since the benchmark uses randomly generated graphs, the exact position of the source, destination, and connecting edges can influence how much of the graph Dijkstra needs to explore. To reduce the effect of these cases, multiple repetitions were executed. Even with this variation, A* was faster in all tested configurations, which shows that the heuristic consistently improved the search.

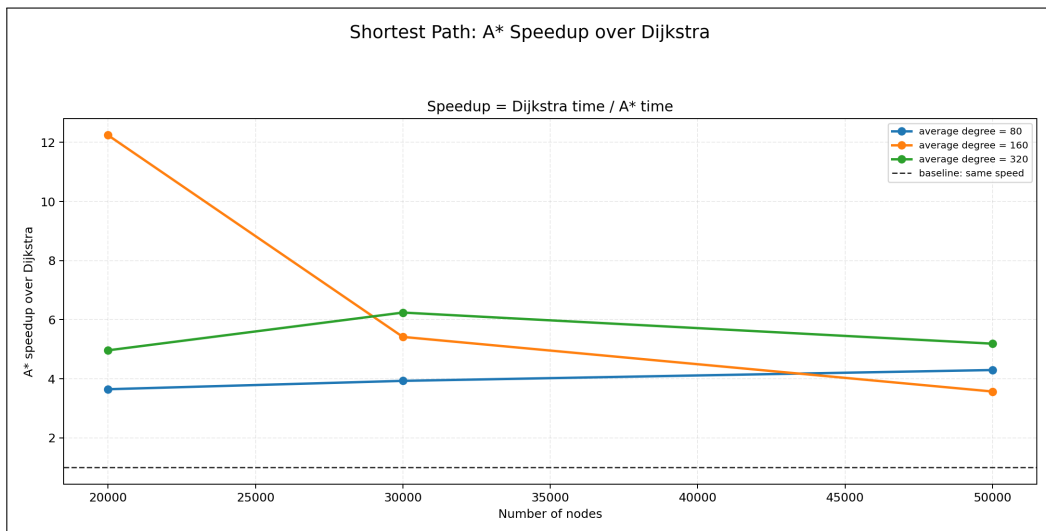


Figure 4: A* speedup over Dijkstra.

To verify that the speedup was related to reduced exploration, we also measured the number of vertices reached by each algorithm. This makes the explanation more concrete: A* is faster because it changes the order in which the graph is explored, prioritizing vertices that are more promising with respect to the destination and avoiding many regions that Dijkstra still has to discover.

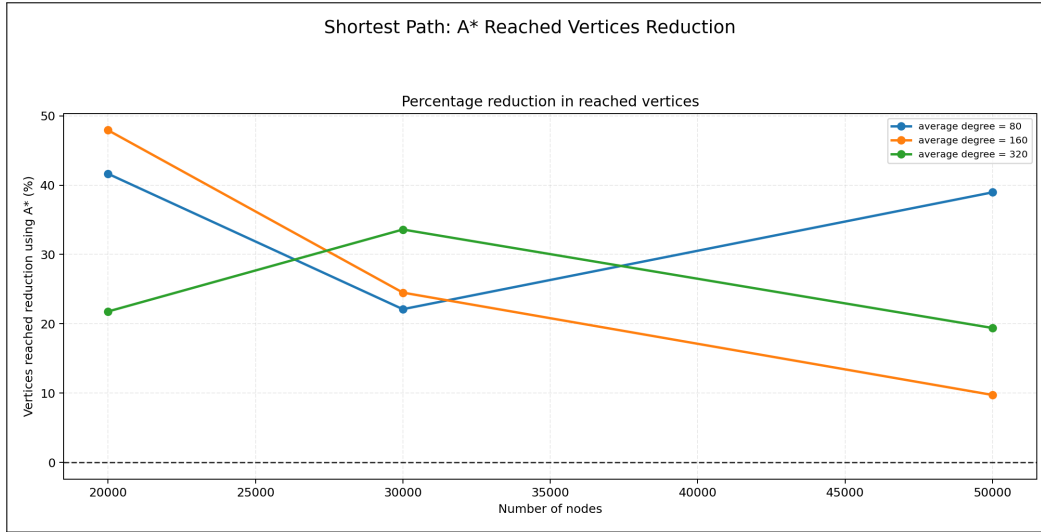


Figure 5: Reduction in reached vertices when using A* instead of Dijkstra.

The reached-vertices reduction plot shows that A* reached significantly fewer vertices than Dijkstra. In the tested configurations, the reduction was often between roughly 20% and 50%. This explains why A* was faster: it did not merely execute the same search more efficiently; it changed the search order so that fewer irrelevant regions of the graph were explored.

6 Priority Queues

Prim’s algorithm repeatedly needs to select the next minimum-priority vertex. Therefore, the priority queue strongly affects runtime. In Prim, the priority queue stores the unvisited vertices ordered by the cheapest edge currently known to connect them to the growing spanning tree.

This project compared three priority queue implementations:

- brute-force priority queue;
- mutable binary heap;
- Fibonacci heap.

The purpose of this comparison was not only to check theoretical complexity, but also to evaluate how each structure behaves in practice. Algorithms and data structures run on real hardware, so memory layout, cache locality, pointer indirection, and implementation overhead can strongly affect the final runtime.

Prim was chosen for this benchmark because it depends heavily on priority queue operations, but it does not depend on a specific source–destination pair like a shortest-path query. In Dijkstra or A*, a benchmark can be affected by accidentally choosing two close vertices or a path that reaches the target early. Prim avoids this issue because it builds a spanning tree over the graph instead of stopping at one destination. Therefore, it gives a more stable way to compare the priority queue implementations. The pseudocode used for Prim is included in Section A.

6.1 Theoretical Comparison

Implementation	Insert	Extract-Min	Decrease-Key
Brute force	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Binary heap	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Fibonacci heap	$\mathcal{O}(1)$ amort.	$\mathcal{O}(\log n)$ amort.	$\mathcal{O}(1)$ amort.

Table 2: Theoretical complexity of priority queue operations.

From a theoretical perspective, the Fibonacci heap is attractive for Prim because **decreaseKey** can be performed in amortized constant time. This is relevant because Prim updates the priority of a vertex whenever it finds a cheaper edge connecting that vertex to the growing spanning tree.

6.2 Brute-Force Priority Queue

The brute-force priority queue stores elements in an unsorted collection. Insertion and **decreaseKey** are simple, but extracting the minimum requires scanning all elements.

Algorithm 3 Brute-Force Extract-Min

Require: Collection Q

```

1:  $best \leftarrow null$ 
2: for all  $x \in Q$  do
3:   if  $best = null$  or  $key[x] < key[best]$  then
4:      $best \leftarrow x$ 
5:   end if
6: end for
7: remove  $best$  from  $Q$ 
8: return  $best$ 
```

This implementation is useful as a correctness baseline, but it is not suitable for large graphs. Since Prim extracts vertices repeatedly while constructing the spanning tree, repeatedly scanning the whole collection quickly dominates the execution time.

6.3 Mutable Binary Heap

The mutable binary heap uses an array-based representation of a complete binary tree. Each vertex also stores its current position inside the heap array, which allows the implementation to locate it directly during a **decreaseKey** operation.

When a vertex priority is decreased, the heap property may be violated with respect to its parent. The implementation restores the heap property by repeatedly swapping the vertex with its parent while its priority is smaller. This keeps **decreaseKey** in $\mathcal{O}(\log n)$ time while preserving the compact memory layout of the binary heap.

Algorithm 4 Binary Heap Decrease-Key

Require: Heap H , vertex v , new key k

```
1:  $key[v] \leftarrow k$ 
2:  $i \leftarrow position[v]$ 
3: while  $i > 0$  and  $key[H[i]] < key[H[parent(i)]]$  do
4:   swap  $H[i]$  and  $H[parent(i)]$ 
5:   update positions
6:    $i \leftarrow parent(i)$ 
7: end while
```

Although the binary heap has logarithmic `decreaseKey`, its implementation is simple and cache-friendly. Because the heap is stored in an array, nearby heap elements are also nearby in memory. This improves cache locality and reduces pointer chasing. In practice, this makes the binary heap both faster and more memory-friendly than a pointer-heavy structure.

6.4 Fibonacci Heap

The Fibonacci heap is a more complex pointer-based structure. It supports cheap insertion and amortized constant-time `decreaseKey`, but it performs consolidation during `extractMin`.

Algorithm 5 Simplified Fibonacci Heap Extract-Min

Require: Fibonacci heap H

```
1:  $z \leftarrow min(H)$ 
2: if  $z \neq null$  then
3:   for all children  $x$  of  $z$  do
4:     add  $x$  to the root list
5:      $parent[x] \leftarrow null$ 
6:   end for
7:   remove  $z$  from root list
8:   consolidate roots with equal degree
9: end if
10: return  $z$ 
```

The Fibonacci heap is therefore theoretically strong, but its pointer-heavy representation can lead to larger constant factors and worse cache locality. Each node needs to store several pointers and structural metadata, and operations often move through linked structures rather than contiguous memory. This makes it an important example of the difference between asymptotic complexity and practical performance.

6.5 Benchmark: Priority Queues

The priority queue benchmark ran Prim with each priority queue implementation. For each graph configuration, the same seed was used for all three implementations, ensuring that the algorithms ran on same graphs. Correctness was checked by comparing the number of visited vertices and selected edges. In the tested cases, all implementations produced matching results.

The first result shows why a real priority queue is necessary. The brute-force implementation was significantly slower than the heap-based implementations.

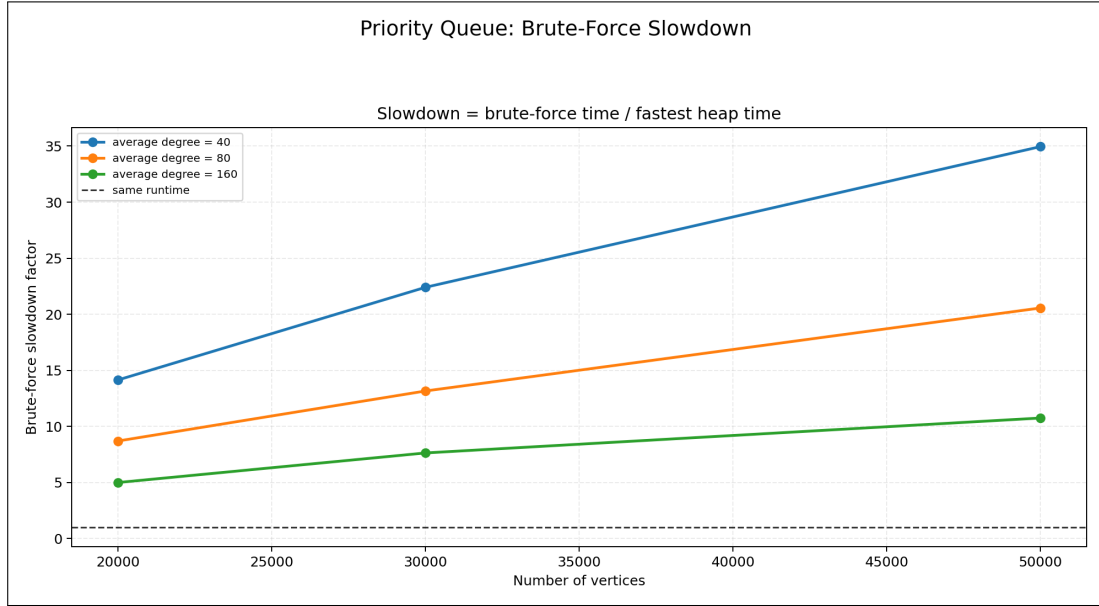


Figure 6: Brute-force priority queue slowdown compared with the fastest heap implementation.

This result is expected from the theoretical analysis. The brute-force priority queue has constant-time insertion, but `extractMin` requires a linear scan. Since Prim repeatedly extracts the next vertex to add to the spanning tree, this becomes very expensive. The slowdown plot shows that the brute-force implementation was much slower than the heap-based implementations, confirming that a naive priority queue is not appropriate for large graph algorithms.

The second result compares the two heap implementations directly.

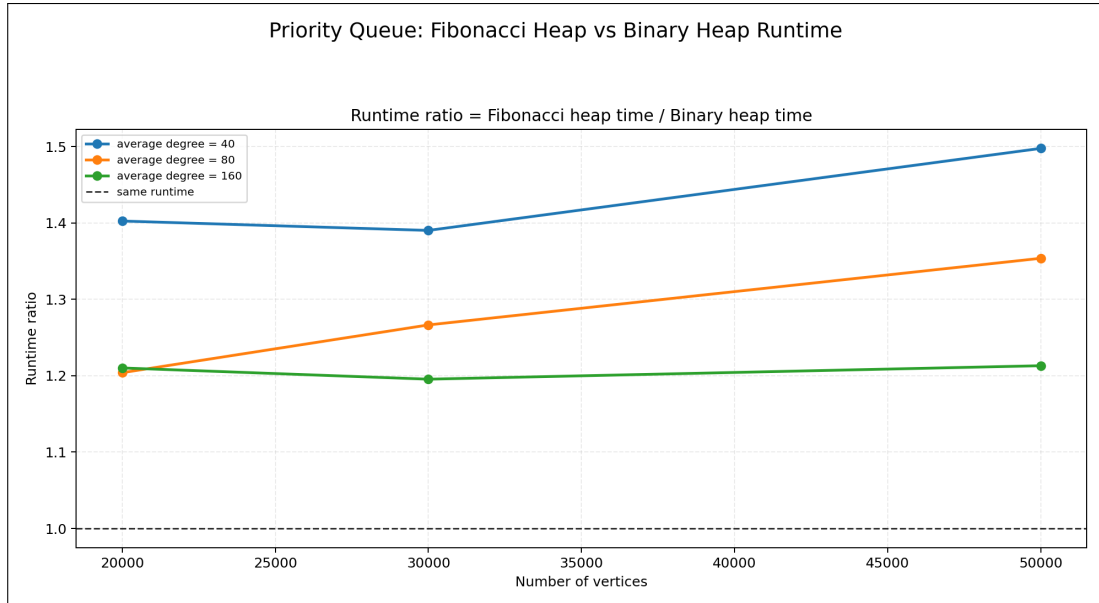


Figure 7: Runtime ratio between Fibonacci heap and binary heap.

The binary heap was consistently faster than the Fibonacci heap in the measured tests. This may seem surprising from the theoretical comparison, because the Fibonacci heap has better amortized complexity for `decreaseKey`. However, the benchmark shows that the cost of pointer-heavy structures can dominate the asymptotic advantage. The binary heap performs more work asymptotically for `decreaseKey`, but the operations are simple and performed over a compact array. The Fibonacci heap performs fewer asymptotic operations, but each operation involves more indirection, more

structural overhead, and less predictable memory access.
This explanation is supported by the cache measurements.

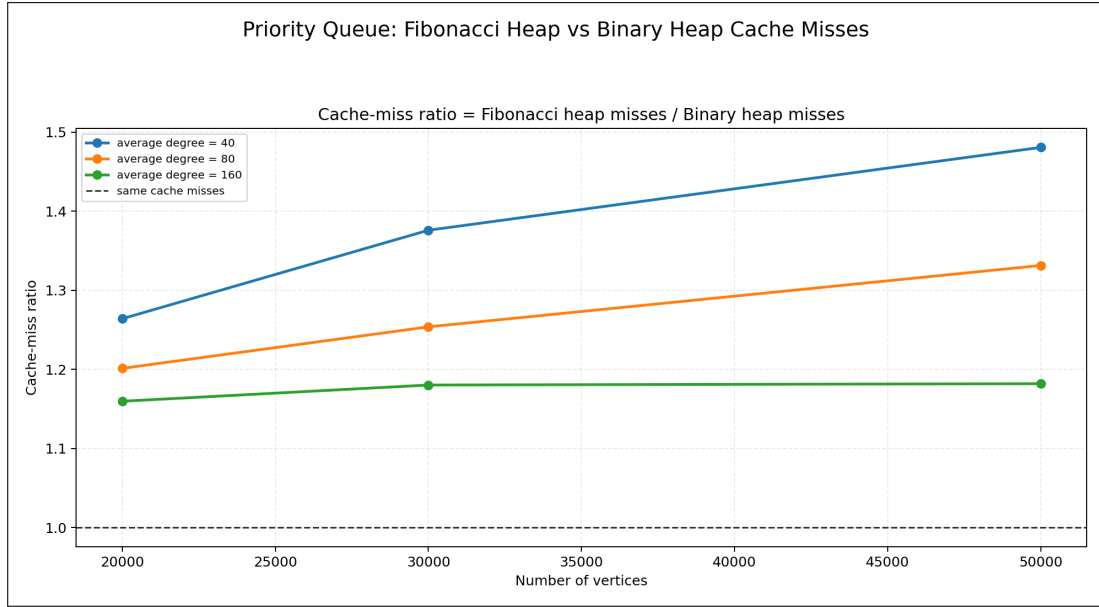


Figure 8: Cache-miss ratio between Fibonacci heap and binary heap.

The Fibonacci heap produced more cache misses than the binary heap. This supports the idea that the binary heap benefits from cache locality, while the Fibonacci heap suffers from pointer indirection. Therefore, the runtime results are consistent with the measured cache behavior.
The result remains consistent across the tested average degrees.

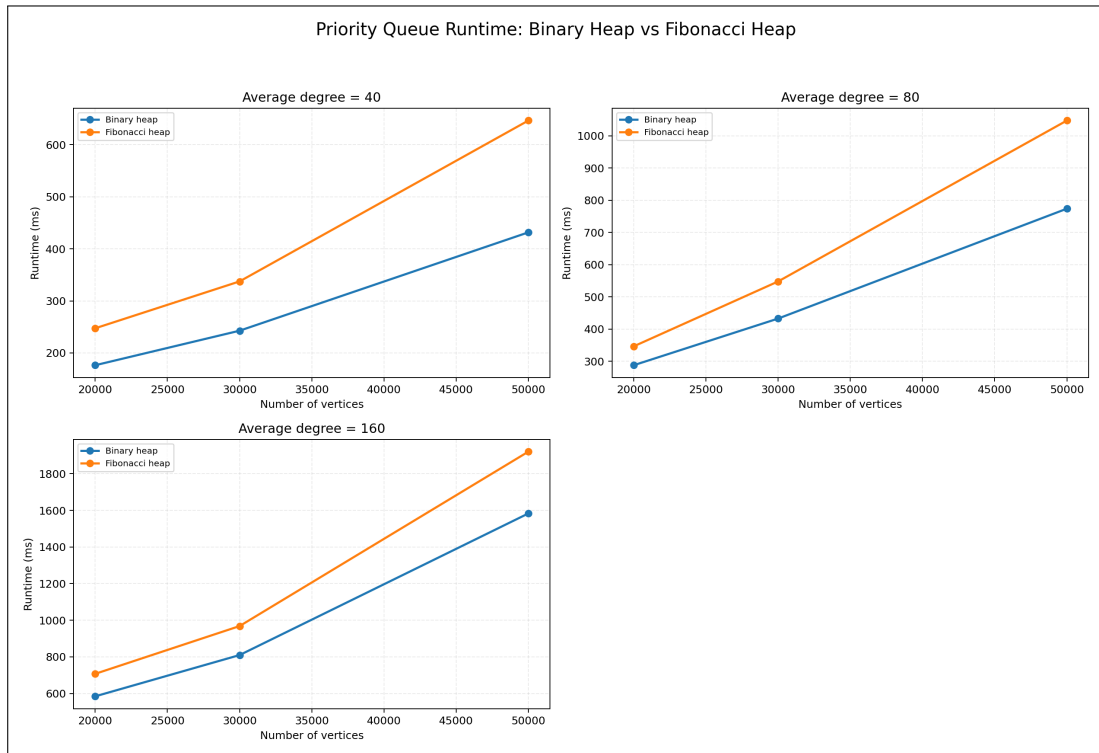


Figure 9: Runtime comparison between binary heap and Fibonacci heap for the tested average degrees.

Increasing the average degree increases the number of candidate edges considered by Prim and can increase the number of priority updates. Even in denser graphs, where the Fibonacci heap should theoretically benefit from cheaper `decreaseKey`, the binary heap remained faster. This reinforces the conclusion that practical runtime is strongly affected by memory layout, cache behavior, and pointer overhead.

Overall, the priority queue benchmark shows three important points. First, a naive brute-force priority queue is too slow for Prim on large graphs. Second, the binary heap was the fastest implementation in practice, despite worse theoretical `decreaseKey` complexity than the Fibonacci heap. Third, the cache measurements support the conclusion that memory layout and hardware behavior can dominate asymptotic advantages.

7 Graph Coloring

Graph coloring assigns a color to each vertex so that no two adjacent vertices share the same color. In this project, coloring was used to study the difference between an exact exponential algorithm and a fast heuristic.

The problem can be mapped to an antenna interference setting:

- each antenna is represented as a vertex;
- an edge connects two antennas if they interfere;
- colors represent frequencies or channels;
- a valid coloring assigns different frequencies to interfering antennas.

The minimum number of colors required is the chromatic number, a NP-hard problem. Computing the chromatic number exactly is computationally expensive, so exact brute force becomes impractical as the graph grows.

7.1 Brute-Force Coloring

Brute-force coloring tries possible assignments and checks if they are valid. If there are n vertices and k possible colors, the number of assignments is:

$$k^n. \tag{3}$$

Algorithm 6 Brute-Force Coloring

Require: Graph $G = (V, E)$, number of colors k

```

1: for all assignments  $c : V \rightarrow \{1, \dots, k\}$  do
2:    $valid \leftarrow true$ 
3:   for all edge  $(u, v) \in E$  do
4:     if  $c[u] = c[v]$  then
5:        $valid \leftarrow false$ 
6:       break
7:     end if
8:   end for
9:   if  $valid$  then
10:    return assignment  $c$ 
11:   end if
12: end for
13: return no valid coloring with  $k$  colors
```

Brute force is useful as a reference for small graphs because it can identify the optimal result. However, it is not suitable for large graphs. Even a small increase in the number of vertices causes a large increase in the number of possible color assignments.

7.2 Welsh–Powell Heuristic

Welsh–Powell is a greedy heuristic. It is fast and simple, but it does not guarantee an optimal coloring. The algorithm first sorts vertices by descending degree, so that highly constrained vertices are colored earlier. Then, for each color, it tries to assign that same color to as many non-adjacent uncolored vertices as possible.

Algorithm 7 Welsh–Powell Coloring

Require: Graph $G = (V, E)$

```

1: compute degree of each vertex
2: sort vertices by descending degree
3:  $color \leftarrow 1$ 
4: while there are uncolored vertices do
5:   choose the first uncolored vertex in the ordered list
6:   assign it  $color$ 
7:   for all remaining uncolored vertices in order do
8:     if vertex is not adjacent to any vertex already using  $color$  then
9:       assign it  $color$ 
10:    end if
11:  end for
12:   $color \leftarrow color + 1$ 
13: end while
14: return coloring

```

7.3 Benchmark: Coloring

The coloring benchmark compared brute-force coloring with Welsh–Powell on small graphs. The input size had to be kept small because brute force grows exponentially. This restriction is itself an important result: the exact algorithm becomes impractical very quickly, even before reaching medium-sized graphs.

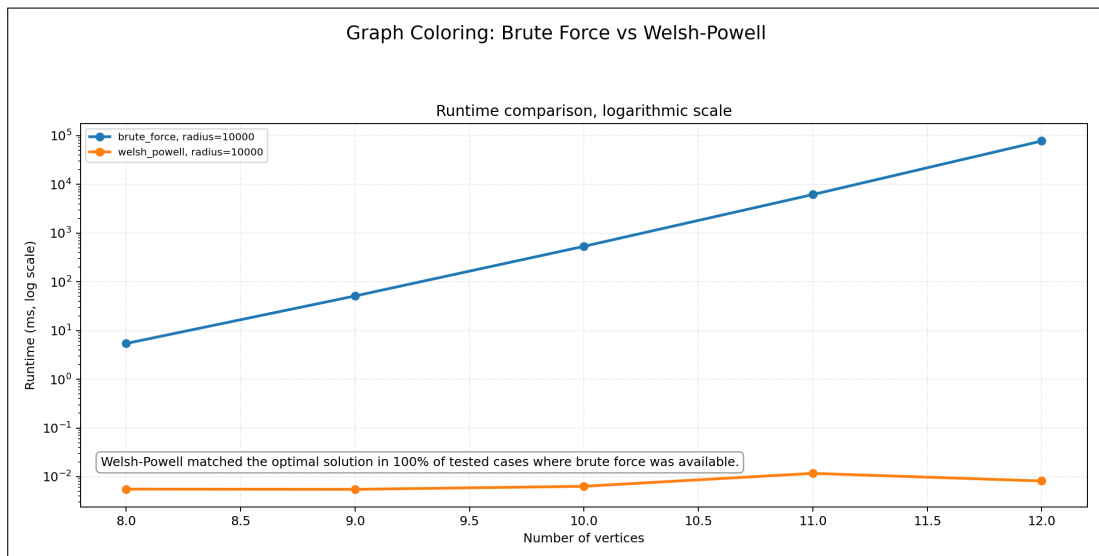


Figure 10: Runtime comparison between brute-force coloring and Welsh–Powell.

The runtime plot shows a clear difference between the two approaches. Even for graphs with only a small number of vertices, brute force is already much slower than Welsh–Powell. The logarithmic y-axis is important: the brute-force curve grows by orders of magnitude, while Welsh–Powell remains almost flat in comparison. This reflects the difference between exhaustive search and a greedy heuristic.

In the tested cases, Welsh–Powell matched the optimal solution found by brute force, so for the tested small graphs, the heuristic produced the same number of colors as the exact algorithm, but in a much shorter time. This does not prove that Welsh–Powell is always optimal; it only means that it was optimal for the tested instances.

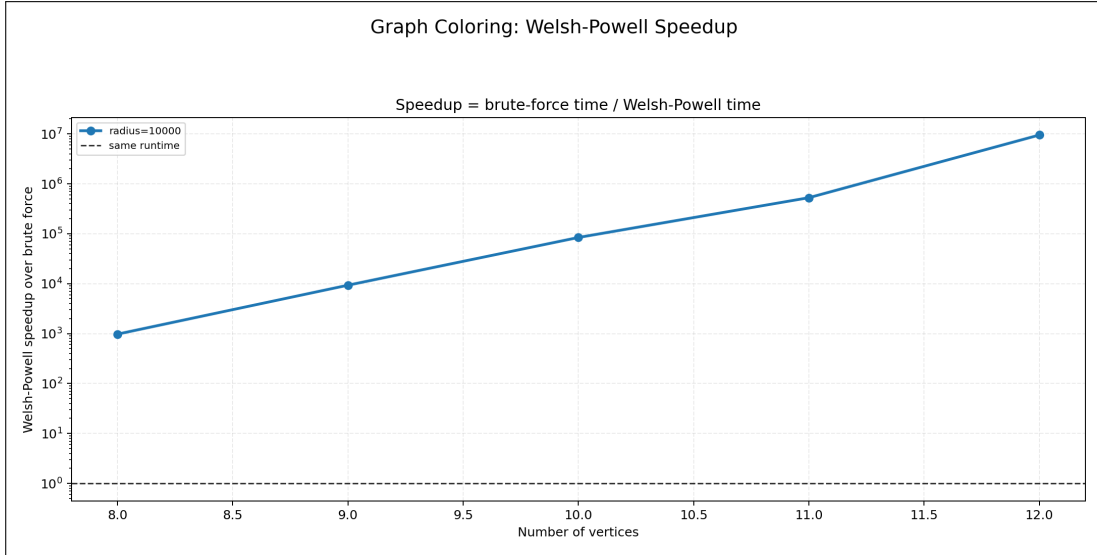


Figure 11: Welsh–Powell speedup over brute force.

The speedup plot makes the practical motivation for heuristics clear. Brute force is valuable as a correctness reference, but it cannot be used as the main approach for larger graphs. Welsh–Powell sacrifices the guarantee of always finding the chromatic number, but it scales much better and still produced optimal results for the tested cases. This illustrates why heuristics are necessary for computationally hard problems: they make it possible to obtain good solutions within a reasonable time.

8 Interference Graph and Quadtree

Coloring requires a graph where edges represent conflicts. In the antenna-interference interpretation, two nodes conflict if they are within a given radius.

$$distance(u, v) \leq r. \quad (4)$$

Therefore, before coloring, an interference graph must be built.

8.1 Brute-Force Interference Graph

The brute-force method checks every pair of points and adds an edge if they are within the interference radius.

Algorithm 8 Brute-Force Interference Graph

Require: Points P , radius r

```
1:  $E \leftarrow \emptyset$ 
2: for  $i \leftarrow 1$  to  $|P|$  do
3:   for  $j \leftarrow i + 1$  to  $|P|$  do
4:     if  $\text{distance}(P_i, P_j) \leq r$  then
5:       add edge  $(i, j)$  to  $E$ 
6:     end if
7:   end for
8: end for
9: return  $(P, E)$ 
```

This approach has $\Theta(n^2)$ pair checks and becomes expensive for large point sets.

8.2 Quadtree-Based Construction

A Quadtree recursively partitions space into quadrants. During a range query, boxes that do not intersect the search radius can be pruned. This avoids unnecessary distance checks.

Algorithm 9 Quadtree Range Search for Interference Graph

Require: Points P , radius r

```
1: build Quadtree  $Q$  from  $P$ 
2:  $E \leftarrow \emptyset$ 
3: for all point  $p \in P$  do
4:    $\text{nearby} \leftarrow Q.\text{rangeSearch}(p, r)$ 
5:   for all  $q \in \text{nearby}$  do
6:     if  $\text{id}(q) > \text{id}(p)$  and  $\text{distance}(p, q) \leq r$  then
7:       add edge  $(p, q)$  to  $E$ 
8:     end if
9:   end for
10: end for
11: return  $(P, E)$ 
```

8.3 Benchmark: Quadtree vs Brute Force

The points are generated uniformly inside a square region of size

$$100000 \times 100000.$$

Two points interfere if their Euclidean distance is at most r . Therefore, each point defines a circular search region of radius r . The larger this circle is, the more points are expected to be inside it, and the harder it becomes for the Quadtree to prune regions.

Radius r	Circle area πr^2	Approx. percentage of total space
1000	3.14×10^6	0.031%
2000	1.26×10^7	0.126%
5000	7.85×10^7	0.785%
10000	3.14×10^8	3.142%

Table 3: Relative size of each interference radius inside the 100000×100000 point-generation space.

Assuming the points are uniformly distributed, the expected fraction of points inside the search circle is approximately

$$\frac{\pi r^2}{100000^2}.$$

Thus, when r is small, only a very small part of the space needs to be searched for each point. This is where the Quadtree is most effective: many quadrants do not intersect the search circle and can be pruned immediately.

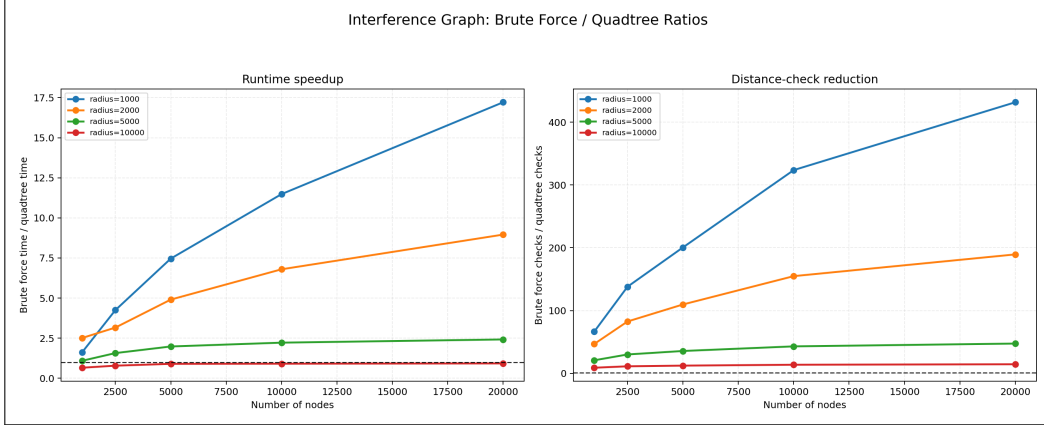


Figure 12: Brute-force over Quadtree ratios for runtime and number of distance checks.

The benchmark shows that the Quadtree is much faster than brute force, especially for smaller radius. For example, with $r = 1000$, the runtime speedup grows significantly as the number of nodes increases. This happens because brute force always checks all pairs of points, giving $\Theta(n^2)$ distance checks, while the Quadtree only checks points that lie in spatial regions near the query point.

The right plot shows the same effect more directly: the number of distance checks is reduced by a large factor when the radius is small. This confirms that the speedup comes mainly from avoiding unnecessary pairwise distance computations.

However, performance decreases as the radius increases. A larger radius creates a larger search region, which means that more Quadtree cells intersect the query area and fewer branches can be pruned. In the extreme case, the query starts to resemble brute force, because most regions must still be visited and checked.

This effect is visible for $r = 10000$. With this radius, each range query covers approximately 3.142% of the total spatial area. Since this is already a relatively large portion of the domain, many Quadtree cells intersect the query region and must still be visited. As a result, the Quadtree can no longer prune enough regions to compensate for the overhead of tree traversal and pointer-based storage, making its performance approach the brute-force method.

9 Discussion

The experiments show that the performance of graph algorithms depends on more than their asymptotic complexity. The same algorithm can behave very differently depending on the data structure used, the structure of the graph, the quality of the heuristic, and the way memory is accessed. Overall, the results support three main ideas: theoretical analysis is necessary but incomplete, heuristics can make otherwise expensive searches practical, and advanced data structures only help when their assumptions match the input.

9.1 Theory vs Practice

The priority queue benchmark is the clearest example of the difference between theoretical and practical performance. From the asymptotic analysis, the Fibonacci heap should be attractive because it provides amortized constant-time `decreaseKey`. This operation is important in Prim, since the priority of a vertex can be updated whenever a cheaper edge to the growing spanning tree is found.

However, the results showed that the mutable binary heap was faster in practice. This happened because the binary heap is implemented using a compact array, which gives it better cache locality and simpler operations. The Fibonacci heap, on the other hand, is pointer-based and requires more structural metadata and pointer traversal. Even if it performs better in the asymptotic model, each operation has a larger practical cost. The cache-miss measurements support this explanation: the Fibonacci heap produced more cache misses than the binary heap, showing that memory layout had a direct effect on runtime.

This does not mean that theoretical analysis is useless. It correctly explains why the brute-force priority queue does not scale: its `extractMin` operation is linear, which becomes extremely expensive when the algorithm repeatedly removes vertices. Instead, the main conclusion is that theory gives expectations, but empirical testing is needed to understand how those expectations behave on real hardware.

9.2 The Role of Heuristics

The experiments also show why heuristics are important. A* and Welsh–Powell use heuristics in different ways, but both demonstrate that adding problem-specific guidance can greatly improve performance.

In the shortest-path problem, A* uses the coordinates of the graph to guide the search toward the destination. Unlike Dijkstra, which only considers the cost already accumulated from the source, A* combines the known cost with an estimate of the remaining distance. When this heuristic is admissible, A* can reduce the explored region without losing optimality. This was visible in the Porto exploration plots: A* was biased toward the destination and reached fewer vertices than Dijkstra, while still finding the same shortest-path distance.

Graph coloring shows a different type of heuristic trade-off. Brute force is exact, but it becomes impractical even for small graphs because the number of possible color assignments grows exponentially. Welsh–Powell avoids this explosion by making a greedy choice based on vertex degree. It does not guarantee the chromatic number in every graph, but it runs quickly and produced optimal results in the tested instances where brute force was still available as a reference. This illustrates a common trade-off in hard problems: when the exact algorithm does not scale, a heuristic may be necessary to obtain a useful solution in reasonable time.

9.3 The Importance of Data Structures

The data structure used to represent and access information had a major impact on the results. Priority queues were essential for making Prim and Dijkstra efficient. The brute-force queue was useful as a baseline, but its linear `extractMin` made it much slower than the heap-based implementations.

The Quadtree experiment showed a similar principle in the spatial part of the project. The Quadtree improves interference graph construction by partitioning space and pruning regions that cannot contain neighbours within the radius. When the radius is small enough, many regions can be eliminated, reducing the number of distance checks and improving runtime. However, the Quadtree is not always better. When the radius becomes too large, or when the data is dense in the same region, most spatial boxes intersect the query area. In that case, the Quadtree approaches brute-force behavior while still paying the overhead of maintaining and traversing the tree. This reinforces the

same lesson from the priority queue benchmark: an advanced data structure is only beneficial when the input allows it to exploit its intended advantage.

10 Conclusion

This project developed and evaluated a multigraph-based prototype for multi-modal routing, together with algorithms for shortest paths, minimum spanning trees, spatial indexing, and graph coloring. The multigraph representation successfully supported multiple transportation modes between the same locations and provided a common structure for the different experiments.

The empirical results showed that no algorithm or data structure is universally superior. Performance depends on the characteristics of the problem, the input data, and the underlying implementation. Visualization also proved useful for understanding algorithm behaviour and motivating improvements, particularly in the comparison between Dijkstra and A*.

Overall, the project demonstrated the importance of combining theoretical analysis with empirical evaluation. Asymptotic complexity provides essential guidance, but practical algorithm design also requires considering implementation details, hardware behaviour, and the structure of the input.

References

- [1] E. W. Dijkstra, *A note on two problems in connexion with graphs*, Numerische Mathematik, vol. 1, pp. 269–271, 1959. doi: [10.1007/BF01386390](https://doi.org/10.1007/BF01386390).
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, *A formal basis for the heuristic determination of minimum cost paths*, IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968. doi: [10.1109/TSSC.1968.300136](https://doi.org/10.1109/TSSC.1968.300136).
- [3] M. L. Fredman and R. E. Tarjan, *Fibonacci heaps and their uses in improved network optimization algorithms*, Journal of the ACM, vol. 34, no. 3, pp. 596–615, 1987. doi: [10.1145/28869.28874](https://doi.org/10.1145/28869.28874).
- [4] D. J. A. Welsh and M. B. Powell, *An upper bound for the chromatic number of a graph and its application to timetabling problems*, The Computer Journal, vol. 10, no. 1, pp. 85–86, 1967. doi: [10.1093/comjnl/10.1.85](https://doi.org/10.1093/comjnl/10.1.85).
- [5] G. Boeing, *OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks*, Computers, Environment and Urban Systems, vol. 65, pp. 126–139, 2017. doi: [10.1016/j.compenvurbsys.2017.05.004](https://doi.org/10.1016/j.compenvurbsys.2017.05.004).
- [6] MobilityData, *General Transit Feed Specification Reference*. Available online: <https://gtfs.org/>.
- [7] Porto Digital, *Open Data Porto*. Available online: <https://opendata.porto.digital/>.

A Prim Pseudocode

Prim’s algorithm was used as an additional benchmark for the priority queue implementations. Unlike Dijkstra, it does not depend on a specific source–destination pair, so it provides a complementary evaluation of heap behavior.

Algorithm 10 Prim with a Priority Queue

Require: Graph $G = (V, E)$, start vertex s , priority queue Q

```
1: for all  $v \in V$  do
2:    $key[v] \leftarrow \infty$ 
3:    $visited[v] \leftarrow false$ 
4:    $parentEdge[v] \leftarrow null$ 
5: end for
6:  $key[s] \leftarrow 0$ 
7:  $Q.insert(s, 0)$ 
8: while  $Q$  is not empty do
9:    $u \leftarrow Q.extractMin()$ 
10:   $visited[u] \leftarrow true$ 
11:  for all edge  $e = (u, v)$  in  $adj[u]$  do
12:    if not  $visited[v]$  and  $weight(e) < key[v]$  then
13:       $key[v] \leftarrow weight(e)$ 
14:       $parentEdge[v] \leftarrow e$ 
15:      if  $v$  is already in  $Q$  then
16:         $Q.decreaseKey(v, key[v])$ 
17:      else
18:         $Q.insert(v, key[v])$ 
19:      end if
20:    end if
21:  end for
22: end while
23: return selected parent edges
```
